Installation of MYSQL

1. Remove an old MySQL installation, if any
First check:
mysql --version

Note : If you are doing a completely fresh installation and don't have anything important in MySQL, you can remove old packages:

sudo apt remove --purge mysql-server mysql-client mysql-common -y
sudo apt autoremove -y
sudo apt autoclean

Then check:

dpkg -l | grep mysql

If this returns nothing relevant, you're starting clean.

Note : Don't remove an existing installation this way if it contains databases you need.

Update Ubuntu/Debian

sudo apt update
sudo apt upgrade -y

Install the tools needed to download/install the repository package:

sudo apt install wget curl gnupg lsb-release -y

3. Download the official MySQL APT Repository package
The official MySQL download page currently provides the repository configuration package:
mysql-apt-config_0.8.39-1_all.deb
You can download it from the official MySQL repository download page:
https://dev.mysql.com/downloads/repo/apt/?utm_source=chatgpt.com
After downloading it, go to your Downloads directory:
cd ~/Downloads
Check the files .
cd ~/Downloads
4. Install the MySQL repository configuration package
Run:
sudo dpkg -i mysql-apt-config_0.8.39-1_all.deb
This opens a configuration screen.
MySQL's official procedure uses this package to add/configure the MySQL APT repository and lets you select the MySQL release series.
In the configuration screen
You'll see something similar to:
MySQL Server & Cluster
MySQL Tools & Connectors
MySQL Preview Packages

Select => MySQL Server & Cluster
5. Update the APT repository
This step is mandatory after adding the MySQL repository.
sudo apt update
You should see MySQL repository information being downloaded.

You can verify the repository:
apt-cache policy mysql-server
You should see MySQL packages available from the MySQL repository.
6. Install MySQL Server
Now install:
sudo apt install mysql-server
The official MySQL APT instructions use this command after configuring the repository.
During installation, you may be asked to configure the MySQL root password depending on the package/version.
Set a strong password and remember it.

---

7. Check MySQL service
The MySQL server normally starts automatically after installation.
Check:
sudo systemctl status mysql
You want to see:
Active: active (running)
If it isn't running:
sudo systemctl start mysql
Enable it at boot:
sudo systemctl enable mysql

---

8. Check MySQL version
mysql --version
For example:
mysql Ver 8.4.x ...

---

9. Run MySQL secure installation
Now run:
sudo mysql_secure_installation
This is the MySQL utility for performing the post-installation security configuration.
Depending on your MySQL version and authentication setup, you'll be asked different questions.
For a typical secure server, you generally want:
Remove anonymous users? Y
Disallow root login remotely? Y
Remove test database and access to it? Y
Reload privilege tables now? Y
If asked about password validation, choose the level appropriate for your requirements; for a personal/lab machine, MEDIUM is a reasonable choice.

---

10. Test the MySQL root login
Try:
mysql -u root -p
Enter the password you created.
If successful:
mysql>
Run:
SHOW DATABASES;
You should see databases such as:
information_schema
mysql
performance_schema
sys
Exit:
exit;

---

11. Find the MySQL executable

Now we come to the environment variable part.
Run:
which mysql
Normally you should get:
/usr/bin/mysql
Check:
ls -l /usr/bin/mysql
The official MySQL APT installation places binaries under /usr/bin and /usr/sbin.
Because /usr/bin is normally already in your PATH, you usually don't need to modify PATH at all.
Check:
echo $PATH
If /usr/bin appears there, you're done with PATH.

---

12. If you specifically need MYSQL_HOME
If your college/practical instructions specifically require a MYSQL_HOME environment variable, you can create it.
First determine the installation location:
ls /usr/bin/mysql
Then open your Bash configuration using gedit:
gedit ~/.bashrc
At the bottom, add:
export MYSQL_HOME=/usr
export PATH=$MYSQL_HOME/bin:$PATH
Save the file.

---

13. Reload the environment variables
Run:
source ~/.bashrc
Check:
echo $MYSQL_HOME
Expected:
/usr
Check PATH:
echo $PATH
And:
which mysql
Expected:
/usr/bin/mysql

---

14. Final verification
Run all of these:
mysql --version
echo $MYSQL_HOME
which mysql
sudo systemctl status mysql
Then:
mysql -u root -p
Inside MySQL:
SELECT VERSION();
Then:
SHOW DATABASES;
Exit:
exit;
6. Install MySQL Server
Now install:
sudo apt install mysql-server
The official MySQL APT instructions use this command after configuring the repository.

During installation, you may be asked to configure the MySQL root password depending on the package/version.
Set a strong password and remember it.

---

7. Check MySQL service
The MySQL server normally starts automatically after installation.
Check:
sudo systemctl status mysql
You want to see:
Active: active (running)
If it isn't running:
sudo systemctl start mysql
Enable it at boot:
sudo systemctl enable mysql

---

8. Check MySQL version
mysql --version
For example:
mysql Ver 8.4.x ...

---

9. Run MySQL secure installation
Now run:
sudo mysql_secure_installation
This is the MySQL utility for performing the post-installation security configuration.
Depending on your MySQL version and authentication setup, you'll be asked different questions.
For a typical secure server, you generally want:
Remove anonymous users? Y
Disallow root login remotely? Y
Remove test database and access to it? Y
Reload privilege tables now? Y
If asked about password validation, choose the level appropriate for your requirements; for a personal/lab machine, MEDIUM is a reasonable choice.

---

10. Test the MySQL root login
Try:
mysql -u root -p
Enter the password you created.
If successful:
mysql>
Run:
SHOW DATABASES;
You should see databases such as:
information_schema
mysql
performance_schema
sys
Exit:
exit;

---

11. Find the MySQL executable
Now we come to the environment variable part.
Run:
which mysql
Normally you should get:
/usr/bin/mysql
Check:
ls -l /usr/bin/mysql

The official MySQL APT installation places binaries under /usr/bin and /usr/sbin.
Because /usr/bin is normally already in your PATH, you usually don't need to modify PATH at all.
Check:
echo $PATH
If /usr/bin appears there, you're done with PATH.

---

12. If you specifically need MYSQL_HOME
If your college/practical instructions specifically require a MYSQL_HOME environment variable, you can create it.
First determine the installation location:
ls /usr/bin/mysql
Then open your Bash configuration using gedit:
gedit ~/.bashrc
At the bottom, add:
export MYSQL_HOME=/usr
export PATH=$MYSQL_HOME/bin:$PATH
Save the file.

---

13. Reload the environment variables
Run:
source ~/.bashrc
Check:
echo $MYSQL_HOME
Expected:
/usr
Check PATH:
echo $PATH
And:
which mysql
Expected:
/usr/bin/mysql

---

14. Final verification
Run all of these:
mysql --version
echo $MYSQL_HOME
which mysql
sudo systemctl status mysql
Then:
mysql -u root -p
Inside MySQL:
SELECT VERSION();
Then:
SHOW DATABASES;
Exit:
exit;
Update system
↓
Install wget/curl/gnupg
↓
Download mysql-apt-config .deb
↓
Install mysql-apt-config
↓
Select MySQL 8.4 LTS
↓
sudo apt update

↓
sudo apt install mysql-server
↓
systemctl status mysql
↓
mysql_secure_installation
↓
mysql -u root -p
↓
Find /usr/bin/mysql
↓
Set MYSQL_HOME (if specifically required)
↓
source ~/.bashrc
↓
Verify MySQL